

Project: Multi-Region Disaster Recovery Architecture

Services Used

Amazon VPC, EC2, Bastion Host, Application Load Balancer, Amazon RDS, Security Groups, Internet Gateway, NAT Gateway, Route Tables

Description

This project implements a **multi-region disaster recovery architecture** on AWS where a web application is deployed in Mumbai (primary region) and replicated to Singapore (secondary region).

The database is replicated using **RDS Read Replica**, and the application is deployed in both regions. In case of failure, the replica is promoted and traffic is manually switched to the secondary region.

Definition

A disaster recovery architecture ensures that applications remain available even if the primary region fails by maintaining a backup system in another region.

Scope of the Project

- Deploy backend application in Mumbai
- Configure secure VPC architecture
- Create RDS primary database
- Configure cross-region read replica
- Deploy backend in Singapore
- Implement manual failover
- Test disaster recovery

This Project Covers

- Multi-region architecture
- Cross-region database replication
- Secure VPC setup
- Bastion access model
- Backend deployment
- Load balancing using ALB
- Health checks configuration
- Manual failover testing

Methodology

1. Multi-Region Disaster Recovery (Implemented)

- Primary region: Mumbai

- Secondary region: Singapore
- RDS read replica used for data replication
- Backend deployed in both regions
- Manual failover using replica promotion

2-Alternative Architecture Using Route 53

If a domain name (e.g., myapp.com) is integrated, **Amazon Route 53** can be used to implement automatic failover and provide a single entry point for users.

In this setup:

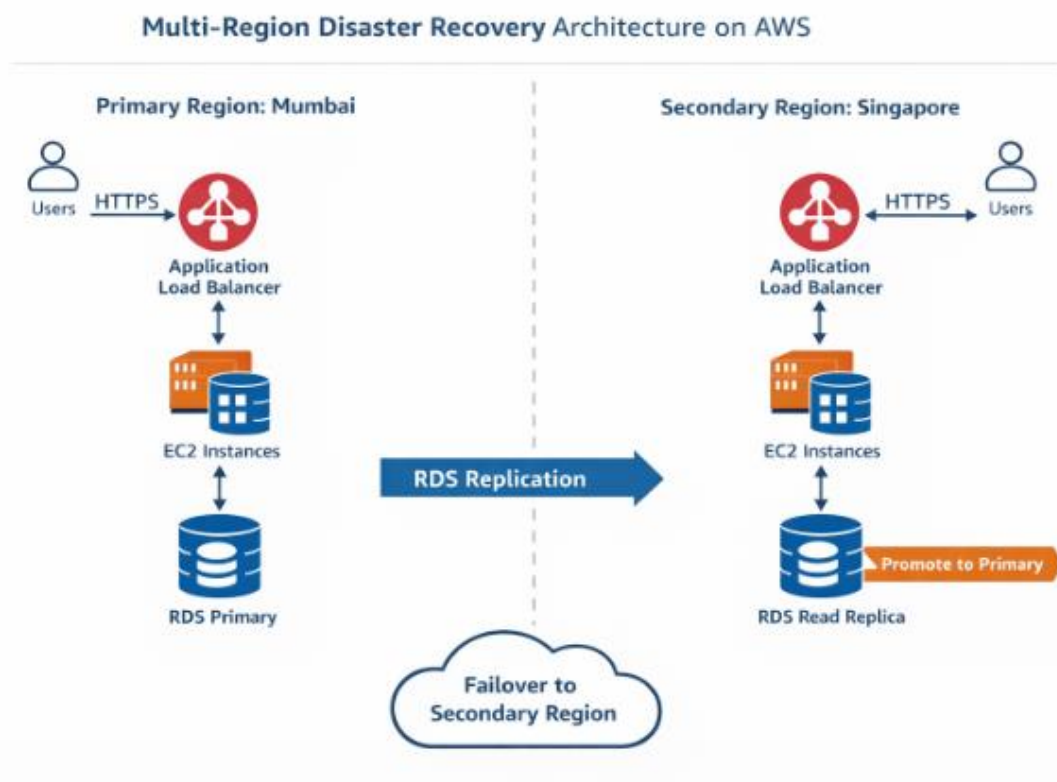
- A **primary record** points to the Mumbai ALB with a health check enabled.
- A **secondary (failover) record** points to the Singapore ALB.

Route 53 continuously monitors the health of the Mumbai application.

If the primary region becomes unavailable, traffic is **automatically redirected** to the Singapore region without manual intervention.

Benefits:

- Single access point (myapp.com)
- Automatic failover based on health checks
- Improved high availability and user experience



AWS Services Used

1. Amazon VPC

Used to create isolated network for both regions.

2. Amazon EC2

Used to run backend application in both Mumbai and Singapore.

3. Bastion Host

Used to securely access private backend EC2.

4. Application Load Balancer

Application Load Balancer (ALB):

- ALB is used to distribute incoming HTTP traffic to backend EC2 instances.
- It performs health checks and sends traffic only to healthy servers.
- It acts as a public entry point for users to access the application.

5. Amazon RDS

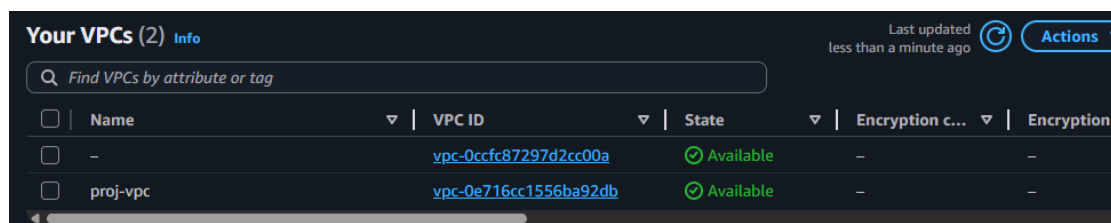
Used as primary database in Mumbai and read replica in Singapore.

Implementation Steps:

Step 1: Create VPC (Mumbai)

Created custom VPC.

- CIDR: 10.0.0.0/16

A screenshot of the AWS Management Console showing the 'Your VPCs (2)' page. The page has a search bar and a table with columns: Name, VPC ID, State, Encryption c..., and Encryption. There are two VPCs listed: one with ID 'vpc-0ccfc87297d2cc00a' and state 'Available', and another named 'proj-vpc' with ID 'vpc-0e716cc1556ba92db' and state 'Available'. The page also shows 'Last updated less than a minute ago' and an 'Actions' button.

☐	Name	VPC ID	State	Encryption c...	Encryption
☐	-	vpc-0ccfc87297d2cc00a	Available	-	-
☐	proj-vpc	vpc-0e716cc1556ba92db	Available	-	-

Purpose:

To create isolated network for project.

Step 2: Create Subnets

Created:

- Public subnet
- Private subnet

Subnets (4) [Info](#) Last updated less than a minute ago [Actions](#) [Cr](#)

Find subnets by attribute or tag

☐	Name	Subnet ID	State	VPC
☐	pvtsub	subnet-09fbac238ddffe471	Available	vpc-0e716cc1556ba92db proj...
☐	pubsub	subnet-001d48bba202581ce	Available	vpc-0e716cc1556ba92db proj...
☐	pvtsub	subnet-0a55f3c3c76cd8b5e	Available	vpc-0e716cc1556ba92db proj...
☐	pubsub	subnet-0be271b2672025c47	Available	vpc-0e716cc1556ba92db proj...

Purpose:

Separate public and private resources.

Step 3: Internet Gateway

- Created IGW
- Attached to VPC

Internet gateways (2) [Info](#) [Actions](#) [Create intern](#)

Find internet gateways by attribute or tag

☐	Name	Internet gateway ID	State	VPC ID
☐	-	igw-02a39faada91fcc92	Detached	-
☐	my-IGW	igw-076281b121e606034	Attached	vpc-0e716cc1556ba92db proj...

Purpose:

Enable internet access for public subnet.

Step 4: NAT Gateway

- Created NAT in public subnet
- Updated private route table → NAT

NAT gateways (1) [Info](#) [Actions](#)

Find NAT gateways by attribute or tag

☐	Name	NAT gateway ID	Connectivity...	State	State message
☐	my-NAT	nat-098cc4961cd343455	Public	Available	-

Purpose:

Allow private EC2 to install packages.

Step 5: Route Tables

- Public → IGW
- Private → NAT

Route tables (4) [Info](#) Last updated 2 minutes ago [Actions](#)

Find route tables by attribute or tag

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	M
☐	-	rtb-0f91c14efca66f1ea	-	-	Y
☐	Private-route	rtb-00bb54af5f1252769	2 subnets	-	N
☐	public-route	rtb-09d1dfacd57059de3	2 subnets	-	N
☐	-	rtb-07e1d69cc8fe48b94	-	-	Y

Step 6: Security Groups

Bastion SG

- SSH (22) → My IP

Backend SG

- SSH (22) → Bastion SG
- Port 3000 → ALB SG

RDS SG

- 3306 → Backend SG

ALB SG

- HTTP 80 → 0.0.0.0/0

Security Groups (6) [Info](#) [Actions](#) [Export security groups to CSV](#)

Find security groups by attribute or tag

☐	Name	Security group ID	Security group name	VPC ID
☐	-	sg-0d1c71197cd520de9	default	vpc-0ccfe...
☐	-	sg-0b79c79685196e556	rds-SG	vpc-0e71...
☐	-	sg-038448c9bf8445781	alb-SG	vpc-0e71...
☐	-	sg-0c2dbd2c0017994e5	bastion-sg	vpc-0e71...
☐	-	sg-078d29181f3afeaa6	default	vpc-0e71...
☐	-	sg-06d0d7c759d249e5d	backend- SG	vpc-0e71...

Step 7: Launch Bastion EC2

- Public subnet
- Public IP enabled
- Security group mumbai-vpc

Step 8: Launch Backend EC2

- Private subnet
- No public IP
- Security group mumbai-vpc

Instances (2) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

< 1 >

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
☐	bastion-server	i-0dd47f29af3a88da0	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1t
☐	backend-server	i-0a1fed946effa419c	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1t

Step 9: copy project Local → Bastion

```
scp -i awsprojectkey.pem -r aws-project ec2-user@<bastion-ip>:~
```

Copy Project and Key → bastion.

Project Local to bastion:

```
scp -i awsprojectkey.pem -r student-data ec2-user@<bastion-ip>:~
```

Key local to bastion:

```
scp -i awsprojectkey.pem -r awsprojectkey.pem ec2-user@<bastion-ip>:~
```

Connect Local → Bastion:

```
ssh -i awsprojectkey.pem ec2-user@<bastion-ip>
```

Project bastion to backend:

```
scp -i awsprojectkey.pem -r student-data ec2-user@<backend-ip>:~
```

Key bastion to backend:

```
scp -i awsprojectkey.pem -r awskeyprojectkey.pem ec2-user@<backend-ip>:~
```

Connect bastion→ Backend:

```
ssh -i "aws-key.pem" ec2-user@10.0.3.10
```

```
chmod 400 awsprojectkey.pem
```

```
[ec2-user@ip-10-0-4-204 ~]$ ls
awsprojectkey.pem  student-data
[ec2-user@ip-10-0-4-204 ~]$ cd student-data
[ec2-user@ip-10-0-4-204 student-data]$ ls
README.md  backend  frontend
[ec2-user@ip-10-0-4-204 student-data]$ |
```

Step 11: Install Node.js

```
sudo yum update -y
```

```
sudo yum install -y nodejs
```

Step 12: Run Backend

```
cd aws-project/backend
```

```
npm install
```

```
node server.js
```

Step 13: Create RDS (Mumbai)

- Engine: MySQL
- VPC: same project VPC
- Private subnet
- Public access: No
- Security group: RDS SG
- Username: admin

- Password: mudasirbhat

Step 16: Create Database Inside RDS

Connected from backend EC2 using MySQL client.

Installed client:

```
sudo yum install -y mariadb105
```

Connected to RDS:

```
mysql -h <RDS-ENDPOINT> -u admin -p
```

Created database:

```
CREATE DATABASE awsdbs;  
SHOW DATABASES;  
EXIT;
```

Purpose: Creating the RDS instance does not automatically create the application database unless specified.

Step 17: Update Backend Code for RDS

Updated **server.js** database connection:

```
const pool = mysql.createPool({  
  host: '<RDS-ENDPOINT>',  
  user: 'admin',  
  password: 'YOUR_PASSWORD',  
  database: 'data',  
  port: 3306  
});
```

Purpose:: The backend must connect to RDS, not localhost.

Step 18: Health Check Testing:

Tested application on backend EC2:

```
curl http://localhost:3000/health
```

Expected output: OK

Purpose: To check that the application is running properly and to allow ALB to perform health checks.

Step

Step 19: Create Target Group (Mumbai)

Created target group with:

- Target type: Instances
- Protocol: HTTP
- Port: 3000
- Health check path: /health

Registered backend EC2 instance.

Name	ARN	Port	Protocol	Target type
tg-backend	arn:aws:elasticloadbalancing...	3000	HTTP	Instance

Purpose: The ALB forwards traffic to a target group, and health checks ensure traffic only goes to healthy instances.

Step 20: Create Application Load Balancer (Mumbai)

Created ALB with:

- Internet-facing scheme
- Two public subnets
- ALB Security Group
- Listener: HTTP 80
- Forward to created target group

Name	State	Type	Scheme	IP address type	VPC
alb-mumbai	Active	application	Internet-facing	IPv4	vpc...

Purpose::ALB becomes the public entry point for the application and distributes traffic to backend instances.

IMPORTANT FIXES (VERY IMPORTANT STEP)

Step 19: Fix Backend & Frontend Issues

Issue 1: App not reachable

Fix:

```
app.listen(3000, '0.0.0.0')
```

Issue 2: Wrong API path (404 error)

Backend:

```
app.get('/api/students')
```

```
app.post('/api/students')
```

Frontend (FIXED):

```
fetch(`${API}/api/students`)
```

Issue 3: API using localhost ✗

Fix:

```
const API = "http://<ALB-DNS>";
```

Issue 4: Security group issue:

Fix:

Backend SG allows port 3000 from ALB SG

After these fixes: Mumbai app fully working

Opened in browser: <ALB-DNS-NAME>



DISASTER RECOVERY SETUP

Everything same as Mumbai except RDS is read replica + promote during failover"

1 -VPC

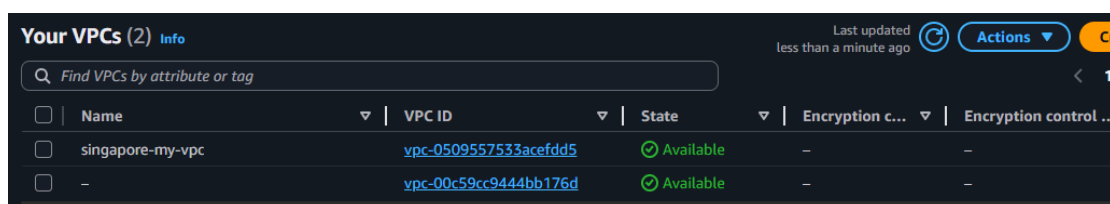
Created **same as Mumbai**

- same CIDR
- public + private subnets

2- IGW + NAT

Same as Mumbai

- IGW for public
- NAT for private



3-Security Groups

Same as Mumbai

- Bastion SG → SSH (My IP)
- Backend SG → SSH (Bastion), Port 3000 (ALB)
- ALB SG → HTTP 80

☐	Name ▾	Security group ID ▲	Security group name ▾	VPC ID
☐	-	sg-0767726e1aa08bff5	backend-SG	vpc-0509557533a
☐	-	sg-09fabb4269a5b90dc	bastion-SG	vpc-0509557533a
☐	-	sg-0c4786571ef203bec	alb-sg	vpc-0509557533a
☐	-	sg-0e3881b4c6b8c7407	default	vpc-0509557533a
☐	-	sg-0ec089bf38cf8fd82	default	vpc-00c59cc9444b
☐	-	sg-0f65b4d6f32334967	rds-SG	vpc-0509557533a

Create RDS Read Replica (Singapore):

Step 1: Create Subnet Group (Singapore) Subnet group defines where RDS will be deployed inside VPC.

Go to:

RDS → Subnet groups → Create

Fill:

- Name: sg-subnet-group
- VPC: Singapore VPC

Add:

- **Private subnets only**

Subnet groups (2) ⌂ Edit Delete Create DB subnet					
Filter by subnet group < 1					
☐	Name	Description	Status	VPC	
☐	default	default	✔ Complete	vpc-00c59cc9444bb176d	
☐	new-singapore-subnet-g	for singapore ds	✔ Complete	vpc-0509557533acefdd5	

Step 2: Go to Mumbai RDS

- Select **primary DB**
- Click **Actions → Create read replica**

Step 3: Configure Replica

- Region → **Singapore**
- DB name → dr-db-singapore
- Subnet group → **sg-subnet-group**

Step 4: Security Group

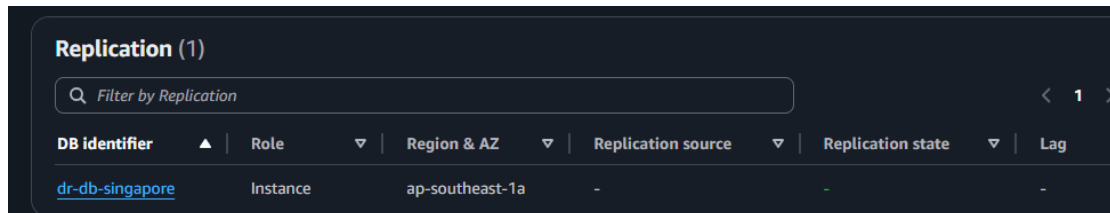
- Select RDS SG

Step 5: Create

Click **Create read replica**

Step 6: Wait

Status = **Available**



The screenshot shows the 'Replication (1)' section in the AWS RDS console. It features a search bar with the text 'Filter by Replication'. Below it is a table with columns: DB identifier, Role, Region & AZ, Replication source, Replication state, and Lag. One row is visible with the DB identifier 'dr-db-singapore', Role 'Instance', Region & AZ 'ap-southeast-1a', Replication source '-', Replication state '-', and Lag '-'.

DB identifier	Role	Region & AZ	Replication source	Replication state	Lag
dr-db-singapore	Instance	ap-southeast-1a	-	-	-

Step 7: Failover (Important)

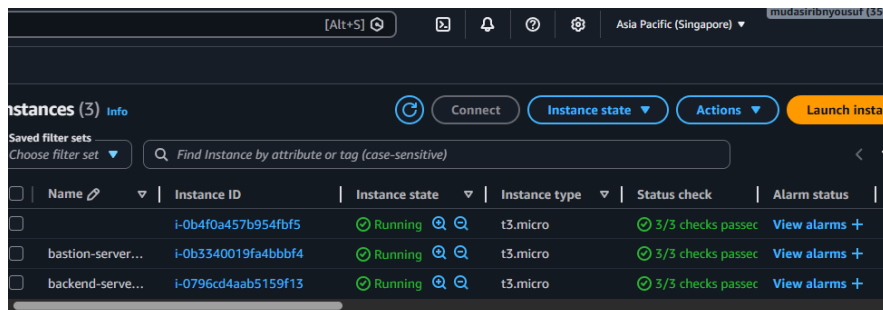
Promote replica:

Convert **read-only replica** → **writable primary DB**.

EC2 Instances:

Same as Mumbai

- Bastion → public subnet
- Backend → private subnet



The screenshot shows the 'Instances (3)' section in the AWS EC2 console. It includes a search bar with the text 'Find Instance by attribute or tag (case-sensitive)'. Below it is a table with columns: Name, Instance ID, Instance state, Instance type, Status check, and Alarm status. Three instances are listed, all with a state of 'Running' and '3/3 checks passed'.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status
	i-0b4f0a457b954fbf5	Running	t3.micro	3/3 checks passed	View alarms +
bastion-server...	i-0b3340019fa4bbb4	Running	t3.micro	3/3 checks passed	View alarms +
backend-serve...	i-0796cd4aab5159f13	Running	t3.micro	3/3 checks passed	View alarms +

Code Deployment

Same process

- local → bastion → backend
- install node
- npm install
- node server.js

RDS (DIFFERENT)

Not created manually

Used **Read Replica from Mumbai**

DB Change

Updated backend:

- DB_HOST → Singapore RDS endpoint

Behavior

- GET → works
- POST → fails (read-only)

Target Group

Same as Mumbai

- Port 3000
- Health /health

ALB

Same as Mumbai

- Internet-facing
- attach backend

Frontend Fix (IMPORTANT)

Same issue as Mumbai

- fixed API path /api/students
- changed API to **Singapore ALB DNS**

FAILOVER (IMPORTANT STEP)

- Promoted read replica
- Replica became primary DB
- Restarted backend
- Application became fully functional

Promote = convert read-only replica → writable primary DB

Test Application

Restart Backend:

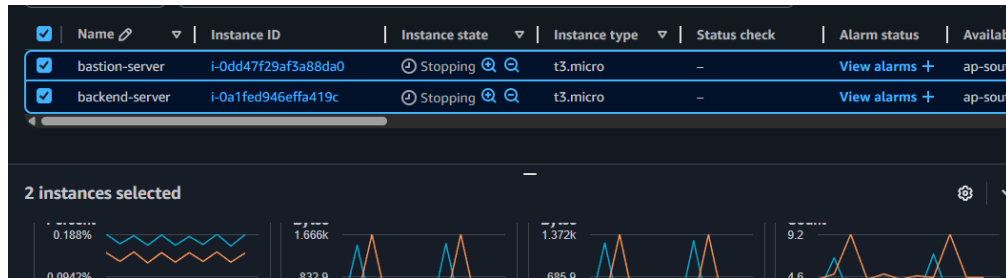
node server.js

curl http://localhost:3000/api/students

Step: Disaster Recovery Testing

Step 1: Simulate Failure

- Stopped Mumbai backend EC2
- Application became unavailable



Step 2: Activate Secondary Region

- Promoted RDS read replica in Singapore
- Restarted backend application

Step 3: Access Application

- Opened Singapore ALB
- Tested:
 - GET request → successful
 - POST request → successful



Troubleshooting:

Issue 1: Target Group Unhealthy

Problem: ALB showing unhealthy targets

Solution:

- Verified /health endpoint exists
- Checked backend SG allows ALB SG on port 3000
- Ensured app listens on 0.0.0.0

Issue 2: 404 Not Found Error

Problem: API routes not working

Solution:

- Backend routes: /api/students
- Updated frontend to match same path

Issue 3: Frontend Calling Wrong API

Problem: Requests going to localhost

Solution:

- Changed API to ALB DNS
- Performed hard refresh

Issue 4: Read Replica Write Error

Problem: Cannot insert data in Singapore DB

Solution:

- Replica is read-only by default
- Promoted replica to make it writable

Conclusion:

This project successfully demonstrates a **multi-region disaster recovery architecture** using AWS services. The application is deployed in Mumbai as the primary region and replicated to Singapore as the secondary region using RDS read replica.

The backend application is deployed in both regions, and Application Load Balancers are used to route traffic. In case of failure, the read replica is promoted to a primary database, and traffic is switched to the Singapore region, ensuring business continuity.

This architecture improves **availability, fault tolerance, and reliability**. The project can be further enhanced by integrating Route 53 for automatic failover and a single entry point.

